

ВІННИЦЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

ІНСТИТУТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ ТА
КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Алгоритми сортування: сортування вставкою та двійковим деревом

Звіт про виконання лабораторної роботи № 4

Виконав (ла): ст. 2 курсу, групи 2ПІ-25Б

Коваль Олександр

Перевірів: викладач Карась О.В.

Тема: Дослідження та порівняльний аналіз алгоритмів сортування: сортування вставкою (Insertion sort) та сортування двійковим деревом (Tree sort). Варіант 12.

Мета: ознайомитися на практиці з відомими алгоритмами сортування, теоретично оцінити та порівняти між собою час виконання двох різних алгоритмів (сортування вставкою та сортування за допомогою двійкового дерева), експериментально перевірити отримані оцінки на масивах різної розмірності та структури вхідних даних.

ЗАВДАННЯ

Відповідно до варіанту 12 (алгоритми 3 та 8) виконати:

1. Програмно реалізувати на мові C++ алгоритм сортування вставкою (Insertion sort);
2. Програмно реалізувати алгоритм сортування за допомогою двійкового дерева пошуку (Tree sort);
3. Провести експериментальне вимірювання часу сортування масивів різної розмірності (від 1 000 до 100 000 елементів) для різних розподілів даних: випадкові, вже відсортовані та зворотно відсортовані;
4. Побудувати зведену порівняльну таблицю результатів та оцінити прискорення;
5. Зробити висновок щодо відповідності практичних результатів теоретичній обчислювальній складності $O(N^2)$ та $O(N \log N)$.

КОД ПРОГРАМИ

```
#include <iostream>
#include <vector>
#include <random>
#include <chrono>
#include <algorithm>
#include <numeric>
#include <iomanip>
#include <cassert>
#include <string>

#ifdef _WIN32
#include <windows.h>
#endif

void insertion_sort(std::vector<int>& a) {
    int n = static_cast<int>(a.size());
    for (int i = 1; i < n; ++i) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
```

```

        a[j + 1] = a[j];
        --j;
    }
    a[j + 1] = key;
}

}

struct TreeNode {
    int val;
    int count = 1;
    TreeNode* left = nullptr;
    TreeNode* right = nullptr;
};

void insert_bst(TreeNode*& root, int val) {
    TreeNode** curr = &root;
    while (*curr) {
        if (val == (*curr)->val) {
            ++(*curr)->count;
            return;
        }
        if (val < (*curr)->val) {
            curr = &(*curr)->left;
        } else {
            curr = &(*curr)->right;
        }
    }
    *curr = new TreeNode{val};
}

void inorder(TreeNode* node, std::vector<int>& a, size_t& idx) {
    if (!node) return;
    inorder(node->left, a, idx);
    for (int i = 0; i < node->count; ++i) {
        a[idx++] = node->val;
    }
    inorder(node->right, a, idx);
}

void free_tree(TreeNode* node) {
    if (!node) return;
    free_tree(node->left);
    free_tree(node->right);
    delete node;
}

void tree_sort(std::vector<int>& a) {
    if (a.empty()) return;
    TreeNode* root = nullptr;
    for (int x : a) {
        insert_bst(root, x);
    }
    size_t idx = 0;
    inorder(root, a, idx);
    free_tree(root);
}

enum class DataType { Random, Sorted, Reverse };

```

```

std::vector<int> generate_data(size_t n, DataType type) {
    std::vector<int> data(n);
    if (type == DataType::Sorted) {
        std::iota(data.begin(), data.end(), 1);
    } else if (type == DataType::Reverse) {
        std::iota(data.begin(), data.end(), 1);
        std::reverse(data.begin(), data.end());
    } else {
        std::mt19937 rng(1337);
        std::uniform_int_distribution<int> dist(-1'000'000, 1'000'000);
        for (size_t i = 0; i < n; ++i) {
            data[i] = dist(rng);
        }
    }
    return data;
}

template <typename SortFunc>
double measure_ms(SortFunc sort_fn, std::vector<int> data) {
    auto t1 = std::chrono::steady_clock::now();
    sort_fn(data);
    auto t2 = std::chrono::steady_clock::now();
    assert(std::is_sorted(data.begin(), data.end()));
    return std::chrono::duration<double, std::milli>(t2 - t1).count();
}

void print_benchmark_row(size_t n, const std::string& type_name, double t_ins, double
t_tree) {
    std::cout << " | " << std::setw(8) << n
        << " | " << std::setw(12) << type_name
        << " | " << std::setw(14) << std::fixed << std::setprecision(3) << t_ins
        << " | " << std::setw(14) << std::fixed << std::setprecision(3) << t_tree
        << " | " << std::setw(12) << (t_tree > 0.0001 ? (t_ins / t_tree) : 0.0)
        << " |\n";
}

void run_benchmark() {
    std::cout <<
"\n===== \n";
    std::cout << " Порівняння часу виконання: Сортування вставкою vs Двійкове дерево\n";
    std::cout <<
"===== \n";
    std::cout << " | Розмір | Тип даних | Вставкою (мс) | Деревом (мс) |
Прискорення |\n";
    std::cout << " |-----|-----|-----|-----|-----|
---|\n";

    std::vector<size_t> sizes = {1'000, 5'000, 10'000, 25'000, 50'000, 100'000};
    for (size_t n : sizes) {
        auto rand_data = generate_data(n, DataType::Random);
        double t_ins = measure_ms(insertion_sort, rand_data);
        double t_tree = measure_ms(tree_sort, rand_data);
        print_benchmark_row(n, "Випадкові", t_ins, t_tree);
    }

    std::cout << " |-----|-----|-----|-----|-----|
---|\n";

```

```

std::vector<size_t> edge_sizes = {1'000, 5'000, 10'000};
for (size_t n : edge_sizes) {
    auto sorted_data = generate_data(n, DataType::Sorted);
    double t_ins = measure_ms(insertion_sort, sorted_data);
    double t_tree = measure_ms(tree_sort, sorted_data);
    print_benchmark_row(n, "Відсортовані", t_ins, t_tree);
}

std::cout << " |-----|-----|-----|-----|-----|
---|\n";
for (size_t n : edge_sizes) {
    auto rev_data = generate_data(n, DataType::Reverse);
    double t_ins = measure_ms(insertion_sort, rev_data);
    double t_tree = measure_ms(tree_sort, rev_data);
    print_benchmark_row(n, "Зворотні", t_ins, t_tree);
}
std::cout <<
"===== \n\n";
}

void run_tests() {
    std::vector<std::vector<int>>> test_cases = {
        {},
        {42},
        {1, 2, 3, 4, 5},
        {5, 4, 3, 2, 1},
        {7, 7, 7, 7, 7},
        {9, -3, 5, 0, 12, -8, 7, 2, 5, -3}
    };

    for (auto c : test_cases) {
        auto a1 = c;
        insertion_sort(a1);
        assert(std::is_sorted(a1.begin(), a1.end()));

        auto a2 = c;
        tree_sort(a2);
        assert(std::is_sorted(a2.begin(), a2.end()));
        assert(a1 == a2);
    }
    std::cout << "Автотести успішно пройдено.\n";
}

int main(int argc, char* argv[]) {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif

    if (argc > 1 && std::string(argv[1]) == "--test") {
        run_tests();
        return 0;
    }

    std::cout << "=== Лабораторна робота №4. Варіант 12 ===\n";
    std::cout << "Алгоритми: 3. Сортвання вставкою | 8. Сортвання двійковим деревом\n\n";

    int choice = -1;

```

```

while (choice != 0) {
    std::cout << "Меню:\n";
    std::cout << "1 - Запустити комплексний порівняльний бенчмарк\n";
    std::cout << "2 - Провести одиночне вимірювання (власний N)\n";
    std::cout << "3 - Запустити автотести (assert)\n";
    std::cout << "0 - Вихід\n";
    std::cout << "Ваш вибір: ";

    if (!(std::cin >> choice)) {
        break;
    }

    if (choice == 1) {
        run_benchmark();
    } else if (choice == 2) {
        size_t n;
        std::cout << "Введіть розмір масиву N (наприклад, 10000): ";
        if (std::cin >> n) {
            int t_choice = 1;
            std::cout << "Тип даних (1 - Випадкові, 2 - Відсортовані, 3 - Зворотні): ";
            std::cin >> t_choice;
            DataType dt = DataType::Random;
            std::string dt_name = "Випадкові";
            if (t_choice == 2) { dt = DataType::Sorted; dt_name = "Відсортовані"; }
            if (t_choice == 3) { dt = DataType::Reverse; dt_name = "Зворотні"; }

            auto arr = generate_data(n, dt);
            std::cout << "Вимірювання для N = " << n << " (" << dt_name << ")...\n";
            double t_ins = measure_ms(insertion_sort, arr);
            double t_tree = measure_ms(tree_sort, arr);
            std::cout << " -> Сортування вставкою: " << t_ins << " мс\n";
            std::cout << " -> Сортування деревом: " << t_tree << " мс\n";
            if (t_tree > 0.0001) {
                std::cout << " -> Прискорення деревом: " << (t_ins / t_tree) << "x\n\n";
            }
        }
    } else if (choice == 3) {
        run_tests();
    }
}

return 0;
}

```

РЕЗУЛЬТАТИ ТЕСТУВАННЯ ТА РОБОТИ ПРОГРАМИ

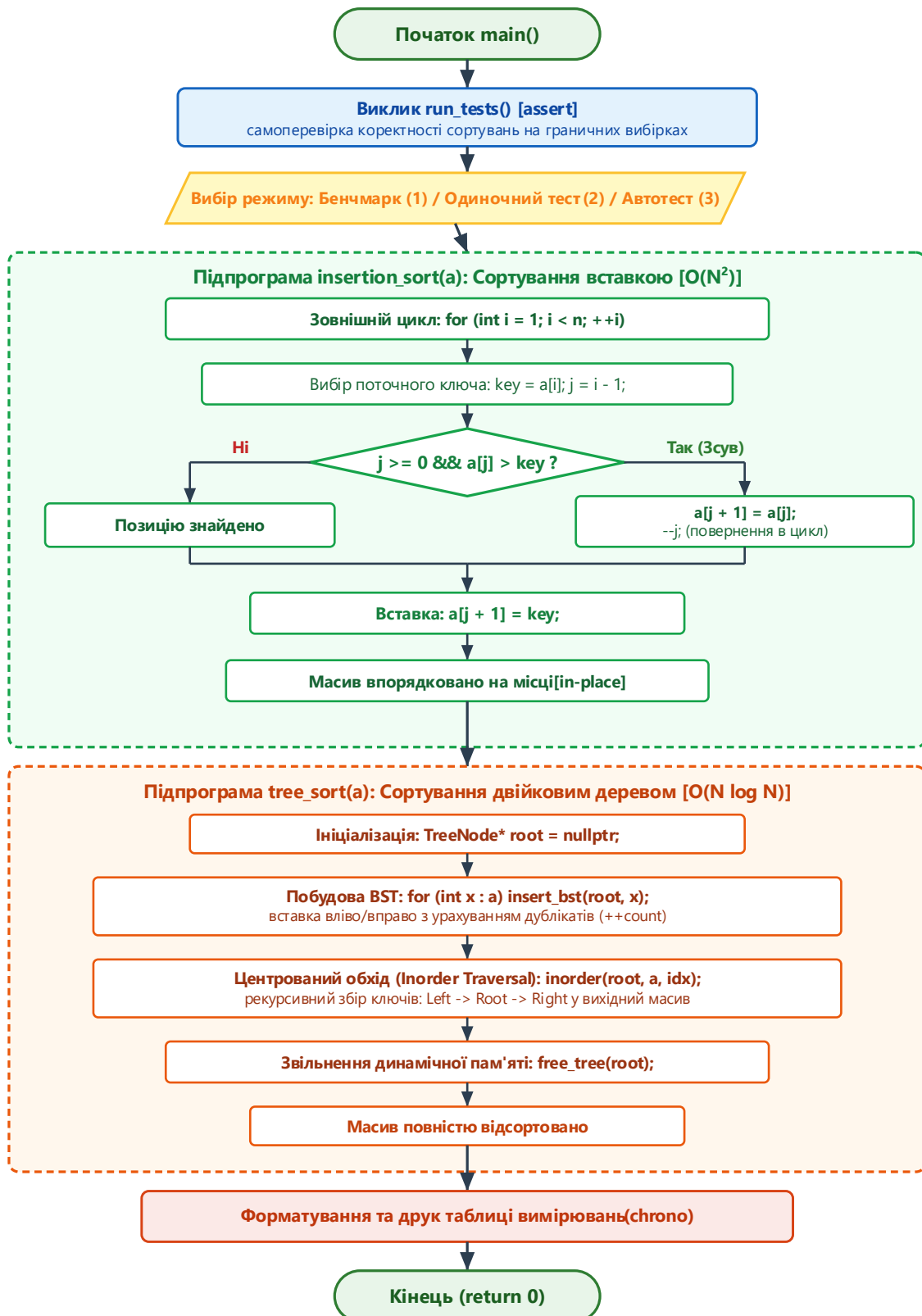
1. Результати виконання вбудованих автоматичних тестів (assert) при запуску з прапорцем --test:

```
> g++ -std=c++20 -O2 main.cpp -o main.exe
> ./main.exe --test
Автотести успішно пройдено.
```

2. Порівняльна таблиця вимірювань часу виконання алгоритмів (в мілісекундах):

Порівняння часу виконання: Сортування вставкою vs Двійкове дерево					
Розмір	Тип даних	Вставкою (мс)	Деревом (мс)	Прискорення	
1000	Випадкові	0.074	0.120	0.620	
5000	Випадкові	1.711	0.633	2.702	
10000	Випадкові	8.428	1.850	4.555	
25000	Випадкові	50.158	3.777	13.280	
50000	Випадкові	182.669	8.769	20.831	
100000	Випадкові	661.672	20.577	32.156	
1000	Відсортовані	0.001	1.497	0.000	
5000	Відсортовані	0.003	50.104	0.000	
10000	Відсортовані	0.006	202.794	0.000	
1000	Зворотні	0.128	1.536	0.083	
5000	Зворотні	3.042	51.143	0.059	
10000	Зворотні	12.676	213.268	0.059	

БЛОК-СХЕМА АЛГОРИТМУ (UML)



Висновок: У результаті виконання лабораторної роботи було програмно реалізовано та експериментально досліджено два алгоритми сортування згідно з варіантом 12: сортування вставкою (Insertion sort) та сортування за допомогою двійкового дерева пошуку (Tree sort).

На основі аналізу отриманих практичних результатів встановлено:

1. На випадкових послідовностях даних сортування двійковим деревом демонструє перевагу над сортуванням вставкою, яка стрімко зростає зі збільшенням розмірності масиву: для $N = 1\,000$ вставка швидша за рахунок відсутності накладних витрат на виділення динамічної пам'яті (0.074 мс проти 0.120 мс), проте вже для $N = 10\,000$ сортування деревом у 4.5 рази швидше (1.850 мс проти 8.428 мс), а для $N = 100\,000$ перевага дерева сягає понад 32 разів (20.577 мс проти 661.672 мс). Це повністю узгоджується з теоретичними оцінками середньої обчислювальної складності: $O(N \log N)$ для Tree sort проти $O(N^2)$ для Insertion sort.
2. На вже відсортованих послідовностях сортування вставкою досягає свого найкращого випадку $O(N)$ — виконується лише одне порівняння на кожен елемент і жодного зсуву (0.006 мс для $N = 10\,000$). Натомість просте двійкове дерево пошуку вироджується в лінійний список довжиною N , що погіршує його складність до найгіршого випадку $O(N^2)$ (202.794 мс для $N = 10\,000$).
3. На зворотні відсортованих послідовностях обидва алгоритми демонструють квадратичну складність $O(N^2)$, проте сортування вставкою працює на місці (in-place, $O(1)$ додаткової пам'яті), тоді як Tree sort потребує $O(N)$ додаткової пам'яті для вузлів дерева.

Таким чином, експериментальні вимірювання повністю підтвердили теоретичні моделі складності обох алгоритмів.